

使用 vLLM 在 Cloud Run 上，透過 RTX 6000 Pro GPU 執行 Gemma 4 模型推論

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/cloud-run-gpu-rtx-pro-6000-gemma4-vllm?hl=zh-tw>

步驟數：12

瞭解如何使用 vLLM，在 Cloud Run 的 NVIDIA RTX Pro 6000 晶片上執行 Gemma 4 推論作業

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 建立服務帳戶](#)
4. [4. 設定 Cloud Storage](#)
5. [5. 擷取及快取模型權重](#)
6. [6. 設定直接虛擬私有雲輸出連線的網路](#)
7. [7. 設定服務帳戶存取權政策](#)
8. [8. 初始化設定變數](#)
9. [9. 部署至 Cloud Run](#)
10. [10. 測試服務](#)
11. [11. 恭喜！](#)
12. [12. 清除所用資源](#)

1. 簡介

注意：這項功能適用《[服務專屬條款](#)》中「一般服務條款」一節的《正式發布前產品條款》。正式發布前功能是依「原樣」提供，支援服務可能受限。詳情請參閱[推出階段說明](#)。

總覽

課程內容

- 如何在 Cloud Run RTX 6000 Pro GPU 上部署 Gemma 4 模型
- 如何使用 [vLLM](#) 和 [Run:ai Model Streamer](#) 加速推論作業，並縮短執行個體啟動時間。

Gemma 4 是 Google DeepMind 開發的 Apache 2 授權開放權重模型系列。這些模型支援多模態和多語言，並提供推論和高效架構。**Cloud Run** 是無伺服器容器環境，支援 GPU。

注意：本程式碼研究室使用 **Gemma 4 31B Instruction-Tuned** 模型。本程式碼實驗室示範如何使用 Run:ai Model Streamer 和直接虛擬私有雲輸出流量，盡量縮短容器啟動期間從 Cloud Storage 載入模型所需的時間。詳情請參閱 [GPU 最佳做法指南](#)。

2. 設定和需求

您可以在指令列執行整個實驗室的内容。您可以使用 Cloud Shell (按一下控制台右上角的提示圖示) 啟動環境。

以下是本程式碼研究室會用到的環境變數。您可以將這些變數儲存在環境檔案中，然後「來源」該檔案。請務必正確設定專案 ID 值，並視需要設定區域。

```
# Model name on HuggingFace Hub
export MODEL_NAME="google/gemma-4-31B-it"

# Cloud Run Service name
export SERVICE_NAME=gemma-rtx-vllm-codelab

# Cloud Project and Region for Cloud Run
export GOOGLE_CLOUD_PROJECT=<YOUR_PROJECT_ID> # Change to your Project Id
export GOOGLE_CLOUD_REGION=europe-west4

# Optional HuggingFace User Access Token for accessing model weights
# (https://huggingface.co/docs/hub/en/security-tokens),
# if you are loading a private model.
export HF_TOKEN=""

# Service account for Cloud Run service
export SERVICE_ACCOUNT="vllm-service-sa"
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT}@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com"

# GCS Bucket for the model cache.
export MODEL_CACHE_BUCKET="${GOOGLE_CLOUD_PROJECT}-${GOOGLE_CLOUD_REGION}-hf-model-cache"
# Model cache location in GSC bucket
export GCS_MODEL_LOCATION="gs://${MODEL_CACHE_BUCKET}/model-cache/${MODEL_NAME}"

# VPC Network for Direct VPC Egress
export VPC_NETWORK="vllm-${GOOGLE_CLOUD_REGION}-net"
export VPC_SUBNET="vllm-${GOOGLE_CLOUD_REGION}-subnet"
export SUBNET_RANGE="10.8.0.0/26"

# set the project
gcloud config set project $GOOGLE_CLOUD_PROJECT
gcloud config set run/region $GOOGLE_CLOUD_REGION
```

啟用本程式碼研究室所需的 API

```
gcloud services enable --project "${GOOGLE_CLOUD_PROJECT}" \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  iam.googleapis.com \  
  compute.googleapis.com \  
  vpcaccess.googleapis.com \  
  storage.googleapis.com
```

步驟 3 · 約 0 分鐘

3. 建立服務帳戶

如果建立 Cloud Run 服務或工作時未指定服務帳戶，Cloud Run 會使用 Compute Engine 預設服務帳戶。建議您為 Cloud Run 服務使用獨立的服務帳戶，避免服務以過多的權限執行。

為 Cloud Run 服務建立服務帳戶

```
gcloud iam service-accounts create ${SERVICE_ACCOUNT} \
  --project "${GOOGLE_CLOUD_PROJECT}" \
  --display-name "vLLM Service Account"
```

步驟 4 · 約 0 分鐘

4. 設定 Cloud Storage

建立 Cloud Storage bucket 來儲存模型權重。這樣一來，Cloud Run 每次啟動服務執行個體時，就能使用直接虛擬私有雲輸出流量，更快下載模型權重。

搭配 vLLM 中的 Run:ai Model Streamer 功能，可大幅縮短模型載入時間。

建立值區

請確認這是與 Cloud Run 服務位於同一位置的單一區域 bucket。

```
gcloud storage buckets create "gs://${MODEL_CACHE_BUCKET}" \
  --uniform-bucket-level-access --public-access-prevention \
  --project "${GOOGLE_CLOUD_PROJECT}" --location "${GOOGLE_CLOUD_REGION}"
```

步驟 5 · 約 0 分鐘

5. 擷取及快取模型權重

接著，將 Gemma 4 模型下載至 Cloud Storage bucket。

模型權重有數十 GB，可能無法下載至本機或 Cloud Shell。

我們改用 Cloud Build，並提供足夠的儲存空間來存放模型權重。

```
gcloud builds submit --project="${GOOGLE_CLOUD_PROJECT}" --region="${GOOGLE_CLOUD_REGION}" --no-source \
    --
substitutions="_MODEL_NAME=${MODEL_NAME},_HF_TOKEN=${HF_TOKEN},_GCS_MODEL_LOCATION=${GCS_MODEL_LOCATION}" \
    --config=/dev/stdin <<'EOF'
steps:
- name: 'gcr.io/google.com/cloudsdktool/google-cloud-cli:slim'
  entrypoint: 'bash'
  args:
  - '-c'
  - |
    set -e
    pip3 install --root-user-action=ignore --break-system-packages huggingface_hub[cli]
    echo "Downloading the model..."
    if [[ "$_HF_TOKEN" != "" ]]; then
      hf download "$_MODEL_NAME" --token $_HF_TOKEN --local-dir "./model-cache/$_MODEL_NAME"
    else
      hf download "$_MODEL_NAME" --local-dir "./model-cache/$_MODEL_NAME"
    fi
    echo "Uploading the model..."
    gcloud storage cp -r "./model-cache/$_MODEL_NAME" "$_GCS_MODEL_LOCATION"
options:
  machineType: 'E2_HIGHCPU_32'
  diskSizeGb: 500
EOF
```

請注意，這會使用相當大的虛擬機器。您也可以改用 `E2_HIGHCPU_8` 執行，甚至完全移除 `machineType` 行。建構時間可能會較長，但也能在配額限制較低的情況下執行。

步驟 6 · 約 0 分鐘

6. 設定直接虛擬私有雲輸出連線的網路

如要設定[直接虛擬私有雲輸出](#)，必須建立網路和子網路，並啟用[Private Google Access](#)。

這樣一來，Cloud Run 服務就能連線至 Google API 和服務 (包括 Cloud Storage) 使用的一組外部 IP 位址。

建立電視網

```
gcloud compute networks create "$VPC_NETWORK" \
  --subnet-mode=custom \
  --bgp-routing-mode=regional \
  --project "$GOOGLE_CLOUD_PROJECT"
```

建立子網路

```
gcloud compute networks subnets create "$VPC_SUBNET" \
  --network="$VPC_NETWORK" \
  --region="$GOOGLE_CLOUD_REGION" \
  --range="$SUBNET_RANGE" \
  --enable-private-ip-google-access \
  --project "$GOOGLE_CLOUD_PROJECT"
```

步驟 7 · 約 0 分鐘

7. 設定服務帳戶存取權政策

Cloud Run 服務帳戶需要權限，才能存取您建立的 Storage Bucket 中的模型權重。

```
gcloud storage buckets add-iam-policy-binding "gs://${MODEL_CACHE_BUCKET}" \  
  --member "serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \  
  --role "roles/storage.admin" \  
  --project "${GOOGLE_CLOUD_PROJECT}"
```

8. 初始化設定變數

定義 vLLM 推論引擎和 Cloud Run 服務的變數。

```
# vLLM variables
export MAX_MODEL_LEN="32767"      # 32767 to improve concurrency. Keep it empty to use model's
maximim context length (256K)
export QUANTIZATION_TYPE="fp8"    # Model quantization for faster performance and lower memory
usage.
export KV_CACHE_DTYPE="fp8"       # KV-cache quantization to save GPU memory.
export GPU_MEM_UTIL="0.95"        # Fraction of GPU memory to be used by the vLLM engine.
export TENSOR_PARALLEL_SIZE="1"   # Partitioning model across GPUs (1 here as we have only 1
GPU).
export MAX_NUM_SEQS="8"           # Max concurrent requests vLLM processes in one batch.

# Cloud Run variables
export CLOUD_RUN_CPU_NUM=20
export CLOUD_RUN_MEMORY_GB=80
export CLOUD_RUN_MAX_INSTANCES=3
export CLOUD_RUN_CONCURRENCY=16
```

效能調整注意事項：調整這些變數時，請在處理量和延遲之間取得平衡：

- **MAX_NUM_SEQS 與 CLOUD_RUN_CONCURRENCY**：CLOUD_RUN_CONCURRENCY 應至少與 MAX_NUM_SEQS 一樣大。如要充分利用資源並兼顧突發流量，請將此值設得稍高 (例如 2 倍)。
- **記憶體壓力**：MAX_MODEL_LEN 和 MAX_NUM_SEQS 都會耗用 GPU 記憶體做為 KV 快取。如果遇到大型內容長度的記憶體不足 (OOM) 錯誤，請考慮減少 MAX_NUM_SEQS。
- **延遲**：並行程度越高 (MAX_NUM_SEQS)，總處理量就越高，但個別要求的延遲時間可能會增加。
- **縮放**：CLOUD_RUN_MAX_INSTANCES 可讓您水平縮放。如果單一執行個體的延遲時間可接受，但您需要更多總容量，請提高這個值。

步驟 9 · 約 0 分鐘

9. 部署至 Cloud Run

準備 vLLM 容器指令列

vLLM 需要大量參數，才能快速有效率地執行大型模型。這些參數會以引數形式傳遞至部署到 Cloud Run 的容器。

```
CONTAINER_ARGS=(
  "vllm"
  "serve"
  "${GCS_MODEL_LOCATION}"
  "--served-model-name" "${MODEL_NAME}"
  "--enable-log-requests"
  "--enable-chunked-prefill"
  "--enable-prefix-caching"
  "--generation-config" "auto"
  "--enable-auto-tool-choice"
  "--tool-call-parser" "gemma4"
  "--reasoning-parser" "gemma4"
  "--dtype" "bfloat16"
  "--quantization" "${QUANTIZATION_TYPE}"
  "--kv-cache-dtype" "${KV_CACHE_DTYPE}"
  "--max-num-seqs" "${MAX_NUM_SEQS}"
  "--gpu-memory-utilization" "${GPU_MEM_UTIL}"
  "--tensor-parallel-size" "${TENSOR_PARALLEL_SIZE}"
  "--load-format" "runai_streamer"
  "--port" "8080"
  "--host" "0.0.0.0"
)

if [[ "${MAX_MODEL_LEN}" != "" ]]; then
  CONTAINER_ARGS+=("--max-model-len" "${MAX_MODEL_LEN}")
fi

export CONTAINER_ARGS_STR="${CONTAINER_ARGS[*]}"
echo "Deployment string: ${CONTAINER_ARGS_STR}"
```

部署 Cloud Run 服務

執行下列指令，部署 Cloud Run 服務。請注意 GPU 類型 (RTX 6000 Pro)、基礎映像檔 (`pytorch-vllm-serve:gemma4`)，以及叫用服務時需要驗證 (`--no-allow-unauthenticated`)。

```

gcloud beta run deploy "${SERVICE_NAME}" \
  --image="us-docker.pkg.dev/vertex-ai/vertex-vision-model-garden-dockers/pytorch-vllm-serve:gemma4" \
  --project "${GOOGLE_CLOUD_PROJECT}" \
  --region "${GOOGLE_CLOUD_REGION}" \
  --service-account "${SERVICE_ACCOUNT_EMAIL}" \
  --execution-environment gen2 \
  --no-allow-unauthenticated \
  --cpu="${CLOUD_RUN_CPU_NUM}" \
  --memory="${CLOUD_RUN_MEMORY_GB}Gi" \
  --gpu=1 \
  --gpu-type=nvidia-rtx-pro-6000 \
  --no-gpu-zonal-redundancy \
  --no-cpu-throttling \
  --max-instances ${CLOUD_RUN_MAX_INSTANCES} \
  --concurrency ${CLOUD_RUN_CONCURRENCY} \
  --network ${VPC_NETWORK} \
  --subnet ${VPC_SUBNET} \
  --vpc-egress all-traffic \
  --set-env-vars "MODEL_NAME=${MODEL_NAME}" \
  --set-env-vars "GOOGLE_CLOUD_PROJECT=${GOOGLE_CLOUD_PROJECT}" \
  --set-env-vars "GOOGLE_CLOUD_REGION=${GOOGLE_CLOUD_REGION}" \
  --port=8080 \
  --timeout=3600 \
  --cpu-boost \
  --startup-probe
tcpSocket.port=8080,initialDelaySeconds=240,failureThreshold=40,timeoutSeconds=10,periodSeconds=15 \
  --command "bash" \
  --args="^;-c;${CONTAINER_ARGS_STR}"

```

部署作業需要幾分鐘才能完成。完成後，您將擁有以 GPU 驅動的環境，透過自動調度資源的無伺服器基礎架構 (包括調度至零，即沒有流量就沒有費用)，提供 Gemma 4 服務。

步驟 10 · 約 0 分鐘

10. 測試服務

部署完成後，您可以使用 vLLM OpenAI 相容 API 與 Gemma 4 模型互動。

取得服務網址

擷取已部署 Cloud Run 服務的網址。

```
SERVICE_URL=$(gcloud run services describe $SERVICE_NAME --project "${GOOGLE_CLOUD_PROJECT}"
--region "${GOOGLE_CLOUD_REGION}" --format 'value(status.url)')
echo "Service URL: $SERVICE_URL"
```

執行推論

使用 `curl` 將提示傳送至模型。

```
curl -s "$SERVICE_URL/v1/chat/completions" \
-H "Authorization: Bearer $(gcloud auth print-identity-token)" \
-H "Content-Type: application/json" \
-d '{
  "model": "'"$MODEL_NAME"'",
  "messages": [
    {"role": "user", "content": "Why is the sky blue?"}
  ],
  "chat_template_kwargs": {
    "enable_thinking": true
  },
  "skip_special_tokens": false
}' | jq -r '.choices[0].message.content'
```

步驟 11 · 約 0 分鐘

11. 恭喜！

恭喜您完成本程式碼研究室！

建議參閱 [Cloud Run](#) 說明文件。

涵蓋內容

- 如何在 Cloud Run RTX 6000 Pro GPU 上部署 Gemma 4 模型
 - 如何設定直接虛擬私有雲輸出流量和 vLLM 模型串流，並搭配 Cloud Storage 加快服務啟動速度。
-

12. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除專案或個別資源。

選項 1：刪除資源

刪除 Cloud Run 服務

```
gcloud run services delete $SERVICE_NAME \
  --project "${GOOGLE_CLOUD_PROJECT}" \
  --region "${GOOGLE_CLOUD_REGION}" \
  --quiet
```

刪除服務帳戶

```
gcloud iam service-accounts delete \
  ${SERVICE_ACCOUNT_EMAIL} \
  --project "${GOOGLE_CLOUD_PROJECT}" \
  --quiet
```

刪除 Cloud Storage bucket

```
gcloud storage rm --recursive gs://${MODEL_CACHE_BUCKET}
```

刪除虛擬私有雲網路和子網路

```
gcloud compute networks subnets delete $VPC_SUBNET \
  --region "${GOOGLE_CLOUD_REGION}" \
  --project "${GOOGLE_CLOUD_PROJECT}" \
  --quiet

gcloud compute networks delete $VPC_NETWORK \
  --project "${GOOGLE_CLOUD_PROJECT}" \
  --quiet
```

方法 2：刪除專案

如要刪除整個專案，請前往「管理資源」，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。如要使用指令列，也可以執行下列指令：

```
gcloud projects delete ${GOOGLE_CLOUD_PROJECT}
```